University OF Engineering and Technology, Lahore
Computer Engineering Department

Course Name: PF	**Course Code:** CMPE-112L
Assignment Type: lab	**Dated: 2 MARCH 2026**
Semester: 1st	**Session: Spring 2026**
Lab/Project/Assignment #: 2	**CLOs to be covered: CLO1**
Lab Title: For Loops	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 2

Lab Goals/Objectives:

• **Eliminate Redundancy:** They keep your code DRY ("Don't Repeat Yourself"), reducing hundreds of lines of repetitive code down to just a few.

• **Iterate Safely:** They provide a structured way to step through sequences like lists, arrays, or ranges without accidentally skipping items or getting stuck in infinite loops.

• **Scale Effortlessly:** The exact same loop structure that processes 5 items can process 5 million items just as easily.

Equipment Required:

• Computer/Laptop
• Python 3.x
• VS Code / PyCharm / IDLE

THEORY:

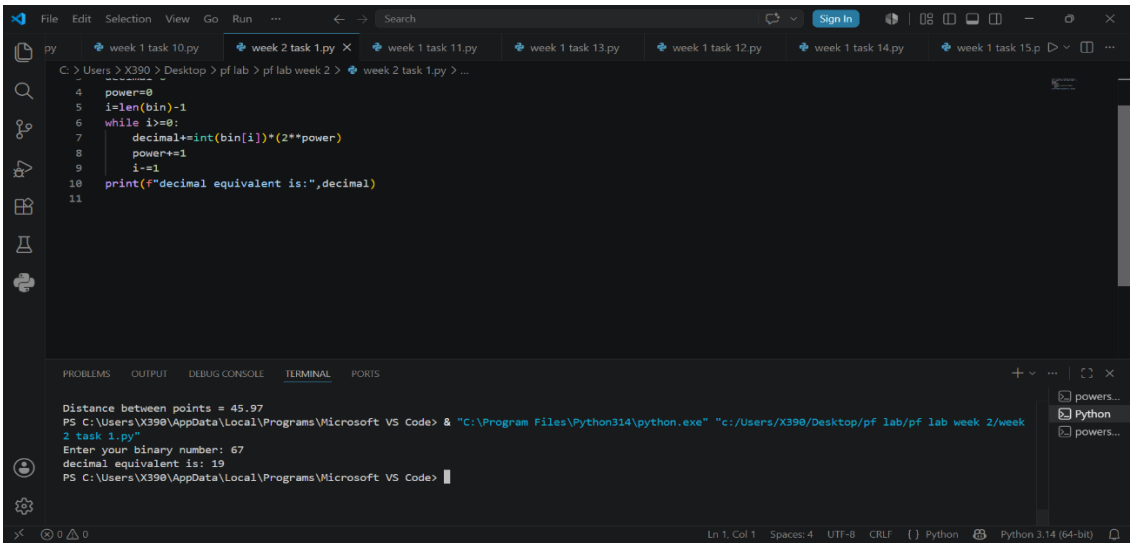
The fundamental theory behind a **`for` loop** is controlled iteration, where a block of code executes repeatedly based on a definitive sequence or counter. Unlike open-ended loops that run until a broad condition changes, a `for` loop is explicitly designed for definite iteration, meaning the number of cycles is usually known before the loop begins. It operates using a tracking variable that automatically advances through a range of numbers or a data structure, like a list or array. Each time the loop completes a cycle, the system evaluates whether there are items left or if the target limit has been reached. Once the collection is exhausted or the counter hits its endpoint, the loop terminates and control flows to the next part of the program. This structured design provides predictable behavior, prevents accidental infinite loops, and forms the mathematical backbone for scanning and filtering large datasets efficiently.

Lab Work:

Task 1:

Write a Python program that takes a binary number (represented as a string) as input from the user and calculates its equivalent **decimal (base-10)** value using a loop, without relying on built-in base conversion functions like `int binary_string, 2)`.

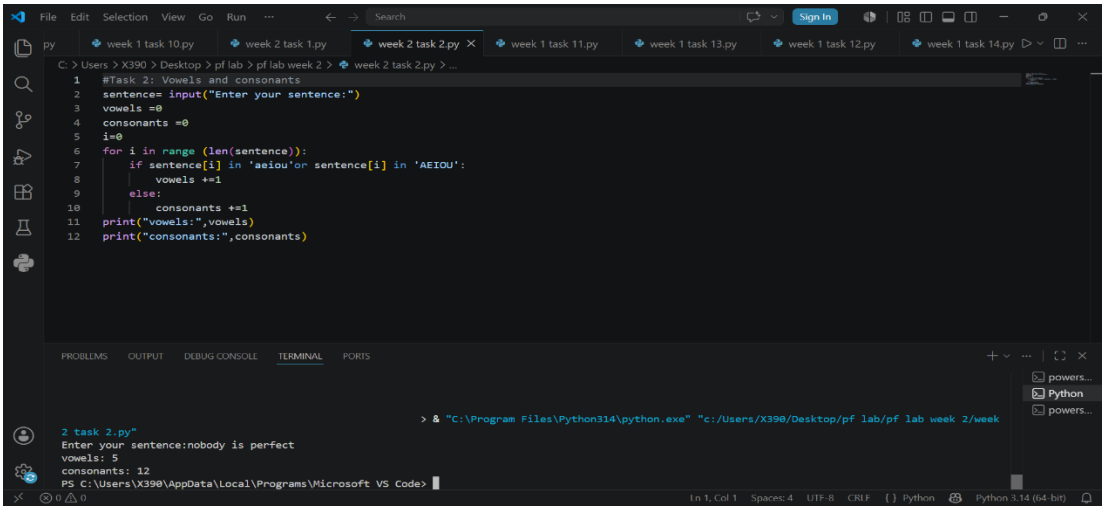
RESULT:



TASK NO 2:

Write a Python program that prompts the user to input a sentence (string) and counts the total number of vowels and consonants present in that sentence using a loop.

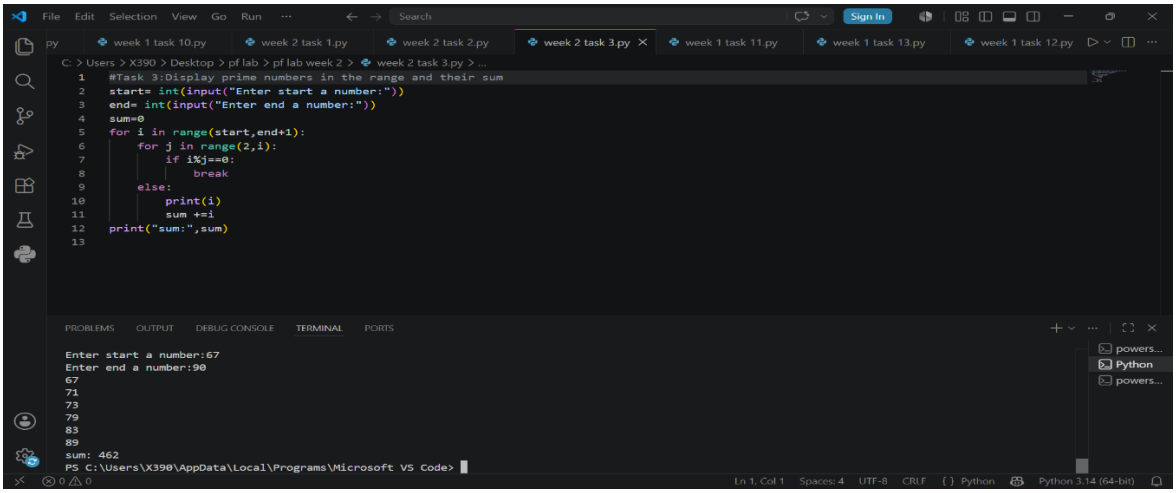
RESULT:



TASK NO 3:

Write a Python program that accepts a numeric range from the user (a starting integer and an ending integer) and utilizes nested loops to find, display, and calculate the cumulative sum of all **prime numbers** within that specified inclusive range.

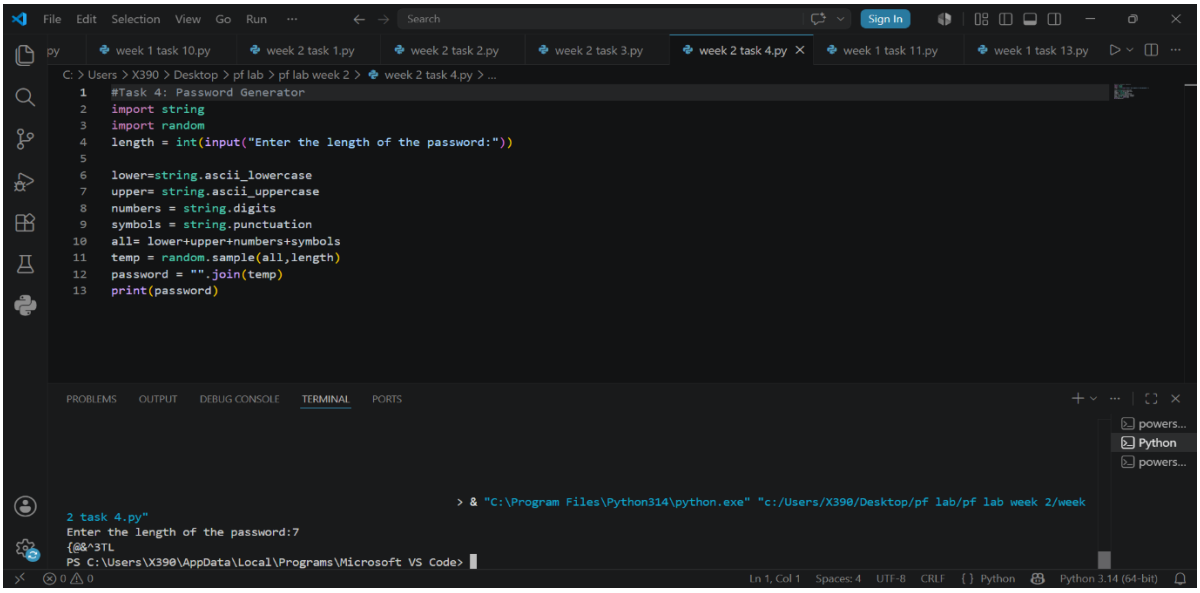
RESULT:



TASK NO 4:

Write a Python program that generates a secure, randomized password based on a user-specified length. The password must combine lowercase letters, uppercase letters, numeric digits, and special punctuation characters to ensure complexity.

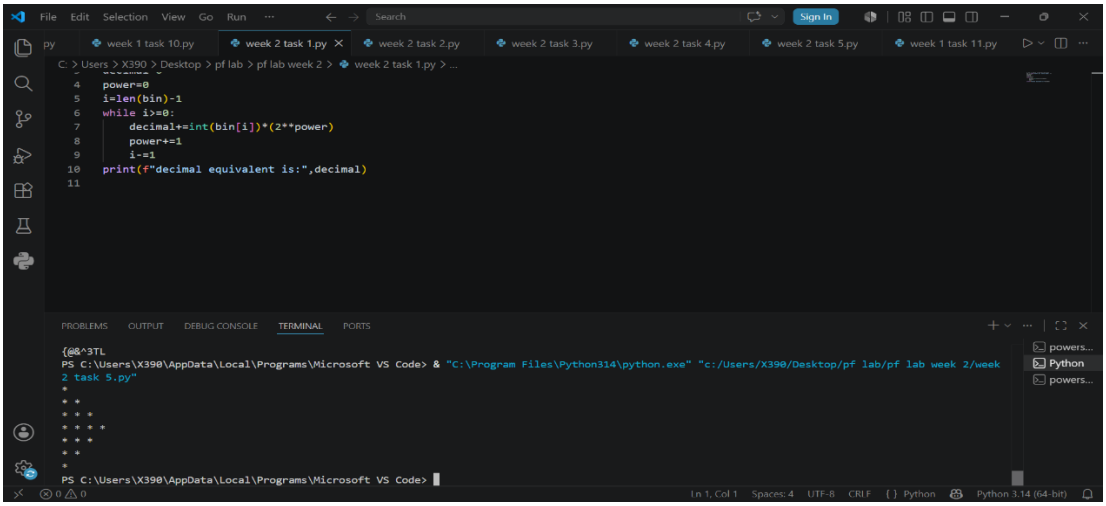
RESULT:



TASK NO 5:

Write a Python program that utilizes loops (such as nested loops or string multiplication) to print a dynamic, side-facing triangular star pattern (*) to the console. The shape increases sequentially to a peak width and then symmetrically decreases back to a single star.

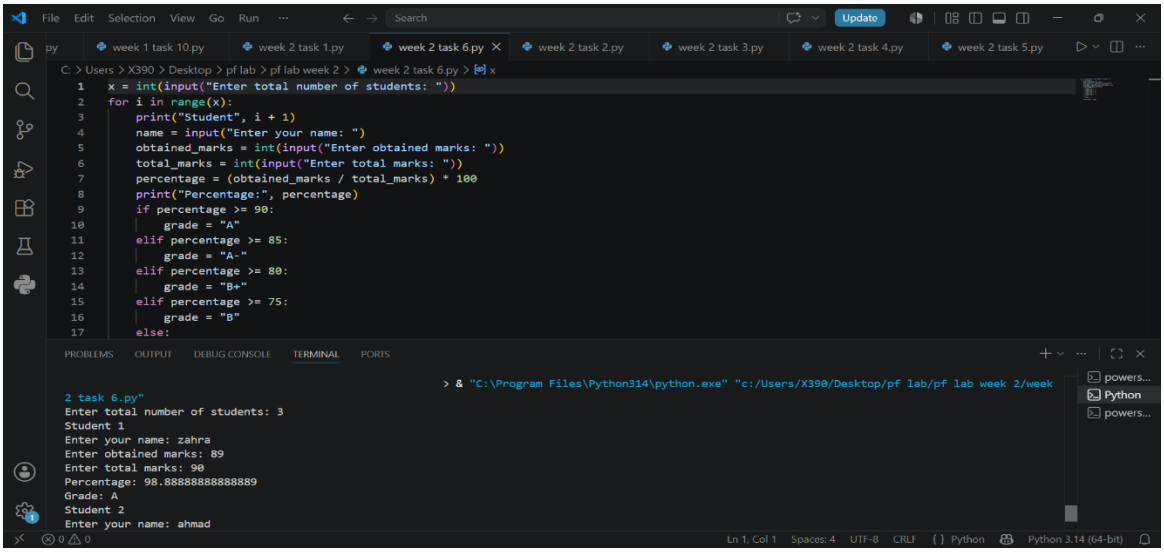
RESULT:



TASK NO 6:

Write a Python program that tracks and processes performance records for a dynamic number of students. The program must loop through a user-defined roster size, prompt for individual academic scores, compute percentage marks, and systematically map those values to an alphabetical grading system

RESULT:



CONCLUSION:

This lab sequence successfully demonstrates core Python programming foundations by using structured control loops to bridge the gap between theoretical math and practical automation. By applying indefinite `while` loops for sequential data scanning and definite `for` loops for character and roster iteration, the tasks highlight how to cleanly manipulate diverse string and numerical inputs. Utilizing nested loops further refines complex problem-solving logic, enabling multi-layered operations like coordinate tracking for star patterns and divisor checks for primality testing. Furthermore, integrating standard modules like `string` and `random` illustrates how to efficiently build secure, automated utilities with minimal overhead. However, the logical edge cases discovered across the tasks—such as a lack of input filtering, untracked string whitespace, and critical indentation slips—powerfully underscore the vital importance of defensive programming. Ultimately, these exercises reinforce how combining smart iteration, conditional mapping, and mathematical formulas allows developers to write scalable, highly efficient code